yabiir benchmark history

Criterion medians for v0.2.0 through v0.7.0

12th Gen Intel(R) Core(TM) i7-12700H, CPU 6 · measured 2026-11-13

1. Summary

Every release from v0.2.0 to v0.7.0 was built and benchmarked one after the other, each pinned to the same core. Most of yabiir got neither faster nor slower: the hot paths were already settled by v0.2.0, and later work went into correctness, observability and new features. Three changes show up clearly in the numbers:

- **v0.6.0, keydir lookups:** the keydir gained a sorted mode and became a two-way `enum`. A hashed `get` went from 9.4 to 11.0 ns, and a miss from 3.9 to 5.1 ns. That is about 1.6 ns per call, for one branch and a larger type.
- **v0.7.0, `read_entry`:** the entry reader stopped trusting the header's length. It reads the body through `take`, growing its buffer as bytes arrive, and feeds the CRC hasher the header and body separately. A small entry costs twice as much (93 ⟶ 168 ns at 64 bytes), while the 64 KiB case got slightly faster. Fixed in v0.8.0, which is faster than v0.6.0 from 1 KiB up and keeps the protection (Section 5.1).
- **v0.7.0, `merge_noop`:** from 4.6 µs to 27.6 ms. This is not a slowdown: the benchmark changed meaning. Merge now seals and compacts the active file, so the "no-op" merge of 50 keys in the active file is a real merge, with its fsyncs.

Everything else in v0.7.0 reads 5–10% slower, including code that did not change at all. Section 2 explains why the conditions, not the code, account for that.

2. How the numbers were taken

`cargo xtask bench` checked each tag out into a worktree, ran `cargo bench` under `taskset` on CPU 6, and kept Criterion's median and its 95% confidence interval for each benchmark. Every tag used the same compiler, the same release profile (`opt-level = 3`, fat LTO, one codegen unit), Criterion 0.8.2 and crc32fast 1.5.1, and each tag ran its own benchmark code. The runs took about three hours in all, from 12:57 to 15:48 UTC, in version order.

The machine was not tuned. Turbo was on and the core ran at its thermal limit (95 °C), which held it near 3.07 GHz, but that clock moves with the room, the fan and the rest of the package. Criterion's intervals cannot see this: the median confidence interval is about ±0.35% for a typical benchmark, while the drift from one run to the next is several percent.

To measure the drift, the report uses a **control index**: the geometric mean, relative to v0.2.0, of the benchmarks whose code is the same in every release. These are `verify_crc`, `encode_entry`, `decode_entry_header`, `decode_hint_header` and `encode_hint`; `encode_entry` gained only a `debug_assert!`, which release builds drop. Figure 1 plots it, and every chart that follows draws it as a dashed grey line. A benchmark that moves with that line is measuring the machine.

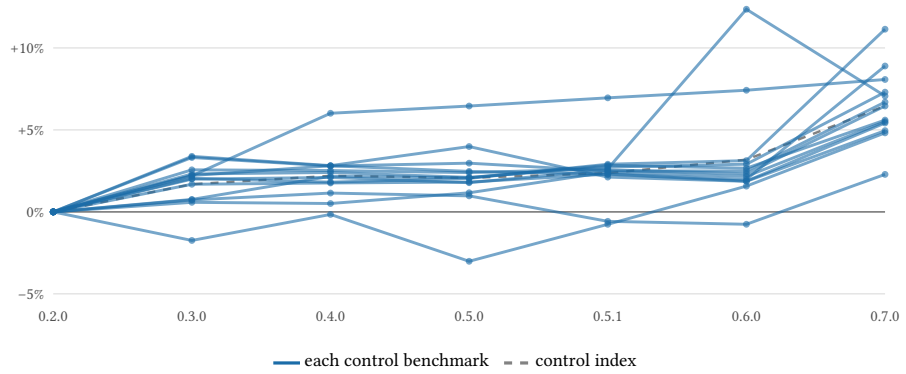


Figure 1: Control benchmarks: unchanged code, relative to v0.2.0. The index climbs steadily, by +3.2% up to v0.6.0 and +6.5% at v0.7.0, which ran last and longest.

So a change of under about 3% between neighbouring releases, or 7% against v0.2.0, should not be read as anything. The redo planned with a fixed clock (a 3 GHz cap, or turbo off) should narrow that.

3. Changes between versions

The commentary for each release lists the changes that could affect speed and what the benchmarks showed.

v0.2.0 (baseline) The engine as first released: the aHash keydir behind one `RwLock`, the append-only data files, recovery from hint files, merge with batched flushes, file locking, and group commit for `sync_on_put`.

v0.3.0 Tests for the gaps cargo-mutants found, including one real merge bug (a live entry after a corrupt one was lost). CI, the `xtask release` command and the docs. **Benchmarks:** flat, apart from the first step of control drift (+1.7%). One `get_miss/1000` outlier in v0.2.0 (7.7 ns, against 3.9 ns for every other size) makes misses look faster from here on.

v0.4.0 Observability: a `Metrics` trait the engine calls on every operation, `tracing` behind a feature, spans in merge, and gauges for merge's pending queue. **Benchmarks:** flat. The keydir benchmarks don't go through the engine, so they can't see the per-operation timing; the merge benchmarks, which do, did not move.

v0.5.0 Hint files gained a whole-file trailing CRC, and recovery falls back to the data file on any fault. Some API tidying. **Benchmarks:** flat; recovery itself isn't benchmarked.

v0.5.1 The crate moved into a workspace. Two concurrency fixes for merge: a reader pins the data file's handle under the keydir lock, and merge waits for in-flight writes (the publish gate). **Benchmarks:** flat. `read_hint` (48.9 ns against 39 ns either side) and `read_entry/64` are one-run outliers.

v0.6.0 The keydir can be kept sorted (a `BTreeMap`), chosen at open, with `Engine::scan` over a key range; a new `keydir_order` benchmark compares the two modes. **Benchmarks:** the hashed keydir pays for the choice: `get_hit` +16.9% and `get_miss` +30.1% against v0.5.1 (Figure 2), from the `match` on every call. `snapshot` from 10,000 keys up is 8–17% slower too, through the iterator `enum`. Inserts, removes and CAS did not move, because their cost is the hashing and the allocation, not the branch.

v0.7.0 The largest release. The single writer became strict (a write is appended and published under one lock), merge seals the active file and reserves its output ids below later writes, a failed fsync fails the writes it covered for good, data-file creation and removal fsync the directory, and a corrupt header can no longer cause an unbounded allocation. The workspace split into six crates, with SQL in its own crate, and the engine, keydir, recovery and merge became module directories. **Benchmarks:** `read_entry` and `merge_noop` changed for the reasons above (Figure 4, Figure 5). The rest moved by 4–10% against v0.6.0, including the control benchmarks (+3.2%). Keydir inserts and removes moved more, by 11–18%, although the keydir code is the same apart from moving the lock wrapper into its own file. That points at the conditions or at code layout, not at a change in the work done, and the redo will settle it.

v0.8.0 (not yet in the history) `read_entry` rejects a header claiming more than the file holds before allocating anything, and hashes the header and body in one call, recovering the small-entry speed (Table 1). Its full run waits for the clock-capped redo.

4. Benchmark families

Each panel shows one benchmark family, a line per parameter (value size in bytes, or key count), relative to its v0.2.0 median, which the panel's header gives. The dashed line is the control index.

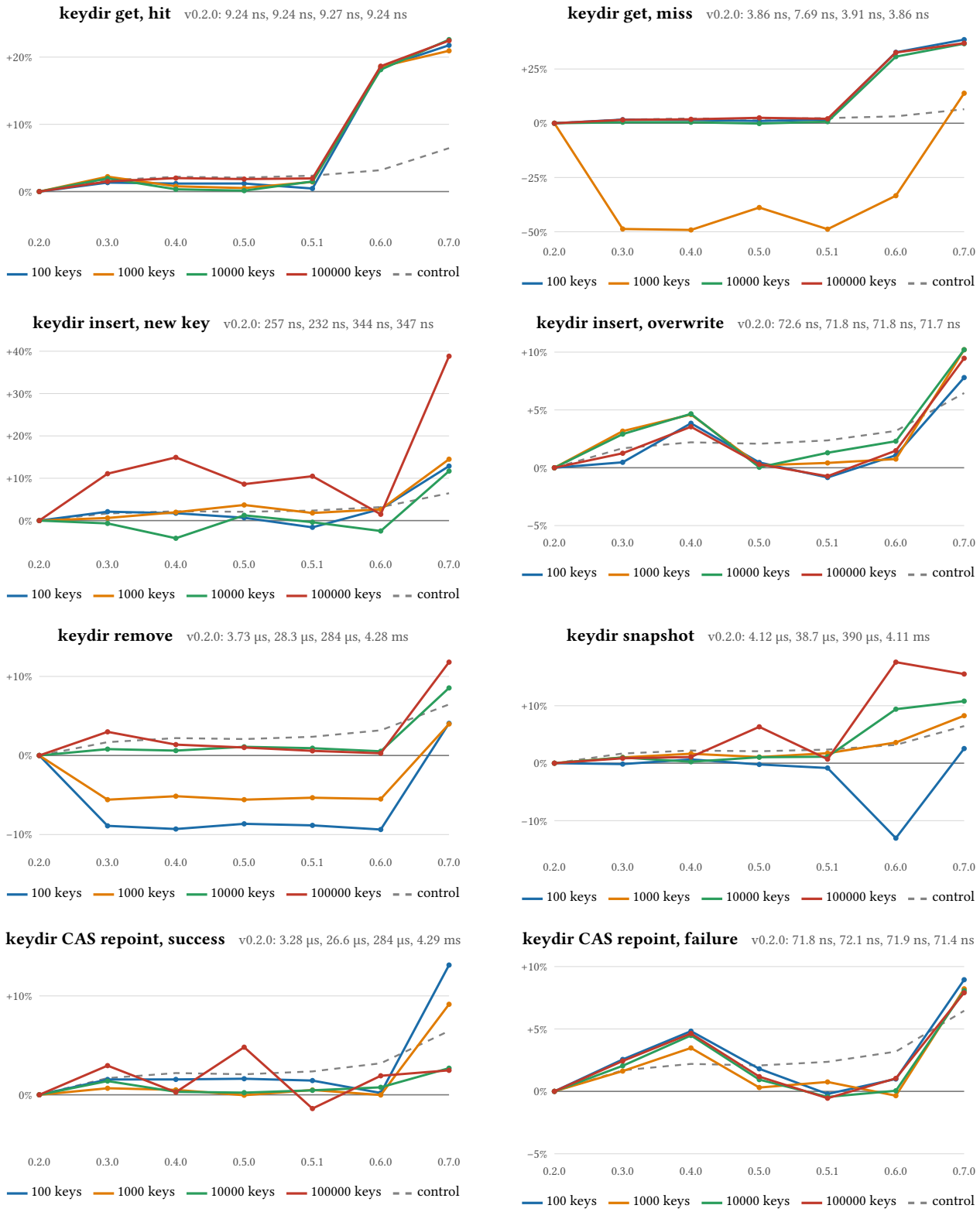


Figure 2: Keydir benchmark families, relative to v0.2.0.

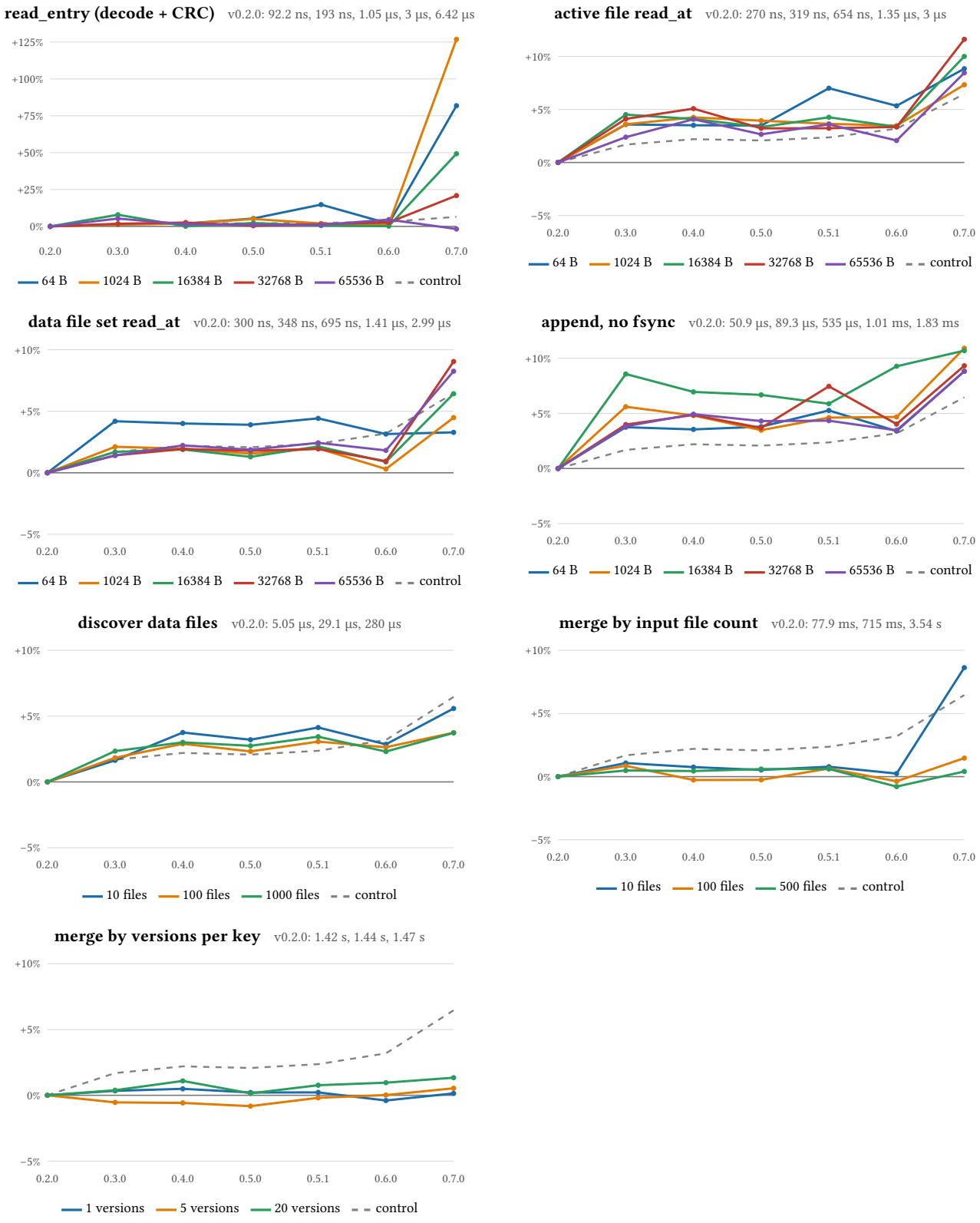


Figure 3: Format, data file and merge benchmark families, relative to v0.2.0.

The keydir panels show the v0.6.0 step in `get` clearly, standing apart from the control line. `get_miss/1000` jumps around (7.7, 3.9, 4.7 and 8.8 ns) while the other sizes sit together. This is the benchmark alternating between two states, probably the key's hash landing in a different cache line, so read the other three sizes. `insert_new_key/100000` is the noisiest benchmark of all, because each sample includes the map growing.

The merge families are the most stable of all: merge spends its time in `write` and `fsync`, which the clock barely affects. They stay within 1% of v0.2.0 up to v0.6.0, except the 10-file merge, which picked up some of the v0.7.0 drift.

5. Two regressions explained

5.1. `read_entry`

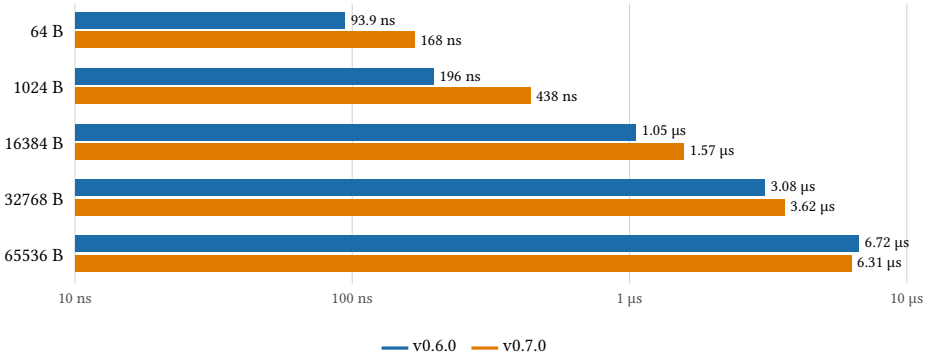


Figure 4: `read_entry` by value size, v0.6.0 against v0.7.0 (log scale).

An entry's lengths come from its header, and the CRC that would vouch for them covers the body too, so they are unverified until the body has been read. A torn or corrupt header can claim about 6 GiB. Up to v0.6.0, `read_entry` allocated whatever the header claimed, then copied the header and body into a second buffer for the CRC. On Linux, overcommit usually lets that allocation succeed; elsewhere it aborts the process, so one torn header could stop a store from opening.

v0.7.0 reads the body through `Read::take(len)` into an empty `Vec`, which grows only as bytes arrive, and feeds a CRC `Hasher` the header and the body separately instead of copying them together. That costs small entries +78.5% at 64 bytes and +123.5% at 1 KiB; at 64 KiB the saved copy wins (−6.1%). `read_entry` is the scanner's inner loop, so recovery and merge of a file of small values pay this per entry.

5.1.1. The fix, in v0.8.0

`read_entry` now takes `available`, the bytes left in the file from the entry's start. A header claiming more is reported as a torn write before anything is allocated, and an honest one gets a single allocation, bounded by the file. The header and body are read into one buffer and hashed in one call.

Each change was measured on its own. Reading into an exactly sized buffer, but still hashing in two calls, left 64-byte entries as slow as v0.7.0. The single buffer, hashed in one call, is what recovered the speed. In a small benchmark outside the crate, `take` with `read_to_end` was the larger cost and the split hash cost only 10–20 ns. Inside the crate, the split-hash version stayed slow, for reasons in how it was compiled that were not pinned down.

value size	v0.6.0	v0.7.0	v0.8.0
64 B	92 ns	168 ns	101 ns
1 KiB	248 ns	594 ns	213 ns
16 KiB	1.45 µs	1.92 µs	1.27 µs
32 KiB	3.88 µs	3.94 µs	3.51 µs
64 KiB	7.93 µs	7.94 µs	7.16 µs

Table 1: `read_entry` medians, measured one after another on CPU 6 on a loaded machine. They are slower than the history runs, so compare across the row, not with the charts.

5.2. `merge_noop`

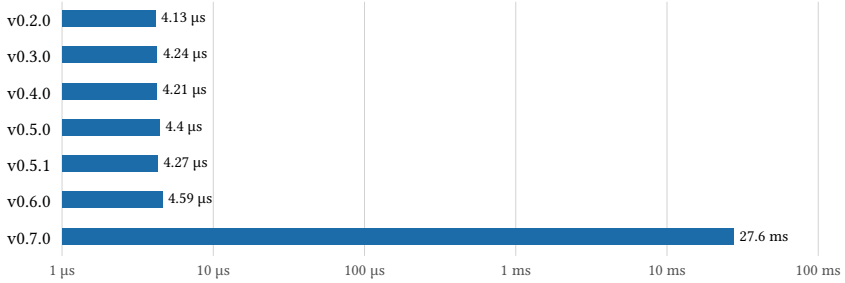


Figure 5: `merge_noop`: a merge of a store whose 50 keys are all in the active file (log scale).

The benchmark writes 50 keys and calls `merge`. Up to v0.6.0, merge left the active file alone; with only the active file there were no inputs, and it returned in about 4 µs. In v0.7.0 merge seals the active file first, so the 50 keys are copied into a merge output, hint file included, and fsynced, and creating and removing files fsyncs the directory. At about 7 ms

per fsync on this disk (`append_with_sync` measures 6.67 ms), 27.6 ms is about four of them. The benchmark should be renamed, or should measure an empty store, to keep a true no-op case.

6. Hashed and sorted keydirs

`keydir_order`, new in v0.6.0, compares the two keydir modes at 1,000, 100,000 and a million keys. Figure 6 shows v0.7.0. Point operations cost 6 to 18 times more in the sorted keydir, and growing from 1,000 to a million keys makes its random lookups 3.4× slower, while a 100-key range is 60,000× faster at a million keys, because the hashed keydir has to filter and sort every key.

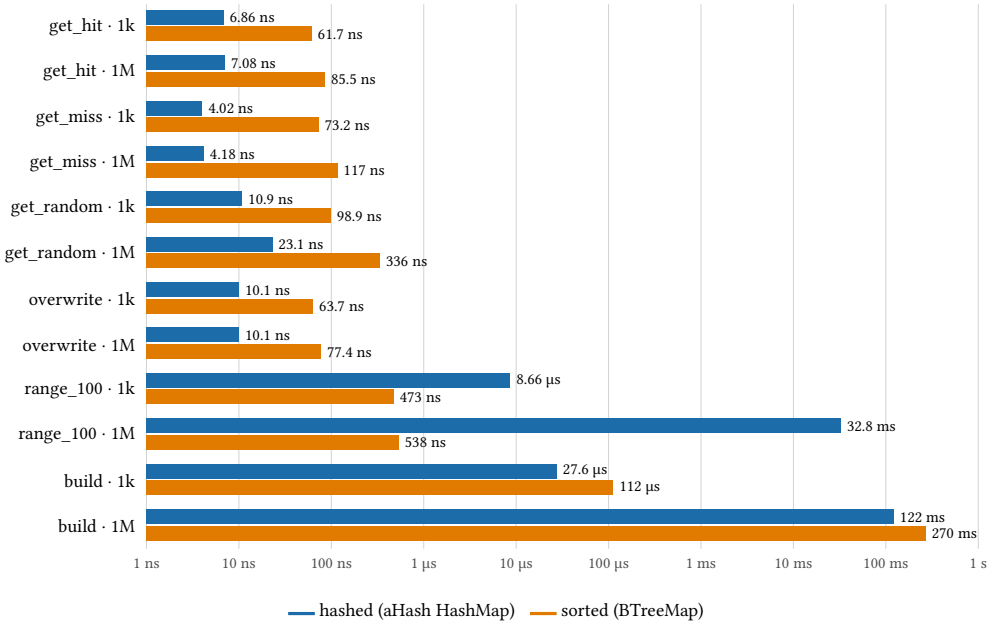


Figure 6: `keydir_order` at v0.7.0, 1,000 and 1,000,000 keys (log scale). `build` is the whole keydir built from scratch, as at open.

Between v0.6.0 and v0.7.0 most of these moved within the drift. A few moved by 10–20% in either direction with no change to the keydir code (shaded in the last table): the builds at 100,000 and a million keys, the hashed range at a million keys and a sorted miss at 100,000 keys. The sorted build at a million keys got slower (+18.7%) while the hashed one got faster (−9.7%). These are long runs dominated by allocation and memory traffic, so no direction among them is a finding yet.

7. All medians

Every median, in Criterion's unit converted for reading. A shaded cell is more than 10% slower (red) or faster (green) than the release before, after taking out the control drift; the `keydir_order` benchmarks exist only from v0.6.0.

benchmark	v0.2.0	v0.3.0	v0.4.0	v0.5.0	v0.5.1	v0.6.0	v0.7.0
active_file_read_at/64	270 ns	280 ns	279 ns	279 ns	289 ns	284 ns	294 ns
active_file_read_at/1024	319 ns	331 ns	333 ns	332 ns	331 ns	330 ns	342 ns
active_file_read_at/16384	654 ns	684 ns	681 ns	676 ns	682 ns	676 ns	719 ns
active_file_read_at/32768	1.35 µs	1.41 µs	1.42 µs	1.4 µs	1.4 µs	1.4 µs	1.51 µs
active_file_read_at/65536	3 µs	3.07 µs	3.12 µs	3.08 µs	3.11 µs	3.06 µs	3.25 µs
append_no_sync/64	50.9 µs	52.8 µs	52.7 µs	52.8 µs	53.5 µs	52.6 µs	55.3 µs
append_no_sync/1024	89.3 µs	94.3 µs	93.6 µs	92.4 µs	93.4 µs	93.5 µs	99 µs
append_no_sync/16384	535 µs	581 µs	572 µs	570 µs	566 µs	584 µs	592 µs
append_no_sync/32768	1.01 ms	1.05 ms	1.06 ms	1.05 ms	1.09 ms	1.05 ms	1.11 ms
append_no_sync/65536	1.83 ms	1.9 ms	1.92 ms	1.91 ms	1.91 ms	1.89 ms	1.99 ms
append_with_sync/1024	6.95 ms	6.96 ms	6.94 ms	6.94 ms	7 ms	6.95 ms	6.67 ms
cas_repoint_failure/100	71.8 ns	73.7 ns	75.3 ns	73.1 ns	71.7 ns	72.5 ns	78.3 ns
cas_repoint_failure/1000	72.1 ns	73.3 ns	74.7 ns	72.4 ns	72.7 ns	71.9 ns	78.1 ns
cas_repoint_failure/10000	71.9 ns	73.3 ns	75.1 ns	72.5 ns	71.5 ns	71.9 ns	77.6 ns
cas_repoint_failure/100000	71.4 ns	73.2 ns	74.8 ns	72.3 ns	71.1 ns	72.2 ns	77.1 ns
cas_repoint_success/100	3.28 µs	3.34 µs	3.34 µs	3.34 µs	3.33 µs	3.29 µs	3.71 µs
cas_repoint_success/1000	26.6 µs	26.8 µs	26.7 µs	26.6 µs	26.7 µs	26.6 µs	29 µs
cas_repoint_success/10000	284 µs	288 µs	285 µs	284 µs	285 µs	286 µs	291 µs
cas_repoint_success/100000	4.29 ms	4.42 ms	4.3 ms	4.5 ms	4.23 ms	4.37 ms	4.4 ms
data_file_set_read_at_cached/64	300 ns	312 ns	312 ns	311 ns	313 ns	309 ns	309 ns
data_file_set_read_at_cached/1024	348 ns	355 ns	354 ns	353 ns	354 ns	349 ns	363 ns
data_file_set_read_at_cached/16384	695 ns	707 ns	708 ns	704 ns	710 ns	701 ns	740 ns
data_file_set_read_at_cached/32768	1.41 µs	1.43 µs	1.44 µs	1.44 µs	1.44 µs	1.42 µs	1.54 µs
data_file_set_read_at_cached/65536	2.99 µs	3.03 µs	3.05 µs	3.04 µs	3.06 µs	3.04 µs	3.24 µs
decode_entry_header	1.01 ns	1.03 ns	1.07 ns	1.07 ns	1.08 ns	1.08 ns	1.09 ns
decode_hint_header	1.74 ns	1.76 ns	1.76 ns	1.76 ns	1.73 ns	1.73 ns	1.78 ns
discover/10	5.05 µs	5.14 µs	5.24 µs	5.22 µs	5.26 µs	5.2 µs	5.34 µs
discover/100	29.1 µs	29.6 µs	29.9 µs	29.7 µs	29.9 µs	29.8 µs	30.1 µs
discover/1000	280 µs	286 µs	288 µs	287 µs	289 µs	286 µs	290 µs
encode_entry/64	43.7 ns	44.6 ns	44.9 ns	44.7 ns	44.7 ns	44.5 ns	46 ns
encode_entry/1024	109 ns	113 ns	112 ns	114 ns	112 ns	111 ns	115 ns
encode_entry/16384	447 ns	458 ns	458 ns	457 ns	460 ns	460 ns	480 ns
encode_entry/32768	1.2 µs	1.24 µs	1.23 µs	1.23 µs	1.23 µs	1.23 µs	1.28 µs
encode_entry/65536	2.36 µs	2.42 µs	2.42 µs	2.42 µs	2.42 µs	2.65 µs	2.53 µs
encode_hint	21 ns	20.7 ns	21 ns	20.4 ns	20.9 ns	21.4 ns	22 ns
get_hit/100	9.24 ns	9.37 ns	9.35 ns	9.35 ns	9.28 ns	10.9 ns	11.3 ns
get_hit/1000	9.24 ns	9.45 ns	9.31 ns	9.29 ns	9.37 ns	11 ns	11.2 ns
get_hit/10000	9.27 ns	9.45 ns	9.3 ns	9.28 ns	9.41 ns	10.9 ns	11.4 ns
get_hit/100000	9.24 ns	9.38 ns	9.42 ns	9.41 ns	9.42 ns	11 ns	11.3 ns
get_miss/100	3.86 ns	3.92 ns	3.92 ns	3.9 ns	3.92 ns	5.12 ns	5.35 ns
get_miss/1000	7.69 ns	3.94 ns	3.91 ns	4.7 ns	3.94 ns	5.12 ns	8.76 ns
get_miss/10000	3.91 ns	3.93 ns	3.93 ns	3.9 ns	3.94 ns	5.11 ns	5.34 ns
get_miss/100000	3.86 ns	3.92 ns	3.93 ns	3.96 ns	3.94 ns	5.12 ns	5.28 ns
insert_new_key/100	257 ns	263 ns	262 ns	259 ns	253 ns	264 ns	291 ns
insert_new_key/1000	232 ns	234 ns	237 ns	241 ns	236 ns	238 ns	266 ns
insert_new_key/10000	344 ns	341 ns	329 ns	348 ns	342 ns	335 ns	384 ns
insert_new_key/100000	347 ns	386 ns	399 ns	377 ns	384 ns	353 ns	482 ns
insert_overwrite/100	72.6 ns	73 ns	75.4 ns	73 ns	72 ns	73.4 ns	78.3 ns
insert_overwrite/1000	71.8 ns	74 ns	75.1 ns	71.9 ns	72.1 ns	72.3 ns	79.1 ns
insert_overwrite/10000	71.8 ns	73.8 ns	75.1 ns	71.8 ns	72.7 ns	73.4 ns	79.1 ns
insert_overwrite/100000	71.7 ns	72.6 ns	74.3 ns	71.9 ns	71.2 ns	72.8 ns	78.5 ns
merge_by_input_file_count/10	77.9 ms	78.7 ms	78.4 ms	78.3 ms	78.5 ms	78 ms	84.6 ms
merge_by_input_file_count/100	715 ms	721 ms	713 ms	713 ms	720 ms	712 ms	726 ms
merge_by_input_file_count/500	3.54 s	3.56 s	3.56 s	3.56 s	3.57 s	3.52 s	3.56 s
merge_by_versions_per_key/1	1.42 s	1.43 s	1.43 s	1.43 s	1.43 s	1.42 s	1.43 s
merge_by_versions_per_key/5	1.44 s	1.43 s	1.43 s	1.42 s	1.43 s	1.44 s	1.44 s
merge_by_versions_per_key/20	1.47 s	1.47 s	1.48 s	1.47 s	1.48 s	1.48 s	1.49 s
merge_noop	4.13 µs	4.24 µs	4.21 µs	4.4 µs	4.27 µs	4.59 µs	27.6 ms
read_entry/64	92.2 ns	93.5 ns	94 ns	97.1 ns	106 ns	93.9 ns	168 ns
read_entry/1024	193 ns	196 ns	197 ns	203 ns	197 ns	196 ns	438 ns
read_entry/16384	1.05 µs	1.14 µs	1.05 µs	1.08 µs	1.06 µs	1.05 µs	1.57 µs
read_entry/32768	3 µs	3.05 µs	3.08 µs	3.01 µs	3.04 µs	3.08 µs	3.62 µs
read_entry/65536	6.42 µs	6.76 µs	6.51 µs	6.49 µs	6.49 µs	6.72 µs	6.31 µs
read_hint	38.4 ns	39 ns	38.8 ns	40.8 ns	48.9 ns	39 ns	40.1 ns
remove/100	3.73 µs	3.4 µs	3.39 µs	3.41 µs	3.4 µs	3.38 µs	3.89 µs
remove/1000	28.3 µs	26.7 µs	26.8 µs	26.7 µs	26.8 µs	26.7 µs	29.4 µs
remove/10000	284 µs	286 µs	286 µs	287 µs	287 µs	286 µs	308 µs
remove/100000	4.28 ms	4.41 ms	4.34 ms	4.32 ms	4.31 ms	4.29 ms	4.79 ms
snapshot/100	4.12 µs	4.12 µs	4.15 µs	4.11 µs	4.09 µs	3.58 µs	4.23 µs
snapshot/1000	38.7 µs	39.1 µs	39.3 µs	39.1 µs	39.4 µs	40.1 µs	41.9 µs
snapshot/10000	390 µs	394 µs	391 µs	394 µs	394 µs	427 µs	432 µs
snapshot/100000	4.11 ms	4.14 ms	4.15 ms	4.37 ms	4.13 ms	4.83 ms	4.74 ms
verify_crc/64	19.4 ns	19.7 ns	19.7 ns	19.7 ns	19.8 ns	19.8 ns	21.1 ns
verify_crc/1024	27.5 ns	28.1 ns	28 ns	28.1 ns	28.3 ns	28.4 ns	30.6 ns
verify_crc/16384	293 ns	294 ns	294 ns	296 ns	300 ns	299 ns	309 ns
verify_crc/32768	575 ns	579 ns	587 ns	587 ns	591 ns	590 ns	607 ns
verify_crc/65536	1.18 µs	1.2 µs	1.2 µs	1.2 µs	1.21 µs	1.2 µs	1.25 µs

benchmark	v0.2.0	v0.3.0	v0.4.0	v0.5.0	v0.5.1	v0.6.0	v0.7.0
order/build/hashed/1000	–	–	–	–	–	28.9 µs	27.6 µs

benchmark	v0.2.0	v0.3.0	v0.4.0	v0.5.0	v0.5.1	v0.6.0	v0.7.0
order/build/hashed/100000	–	–	–	–	–	2.84 ms	2.62 ms
order/build/hashed/1000000	–	–	–	–	–	135 ms	122 ms
order/build/sorted/1000	–	–	–	–	–	117 µs	112 µs
order/build/sorted/100000	–	–	–	–	–	19.3 ms	17.3 ms
order/build/sorted/1000000	–	–	–	–	–	227 ms	270 ms
order/get_hit/hashed/1000	–	–	–	–	–	7.1 ns	6.86 ns
order/get_hit/hashed/100000	–	–	–	–	–	6.74 ns	7.24 ns
order/get_hit/hashed/1000000	–	–	–	–	–	6.72 ns	7.08 ns
order/get_hit/sorted/1000	–	–	–	–	–	63.5 ns	61.7 ns
order/get_hit/sorted/100000	–	–	–	–	–	100 ns	112 ns
order/get_hit/sorted/1000000	–	–	–	–	–	87.6 ns	85.5 ns
order/get_miss/hashed/1000	–	–	–	–	–	4 ns	4.02 ns
order/get_miss/hashed/100000	–	–	–	–	–	3.98 ns	4.15 ns
order/get_miss/hashed/1000000	–	–	–	–	–	4.13 ns	4.18 ns
order/get_miss/sorted/1000	–	–	–	–	–	72.7 ns	73.2 ns
order/get_miss/sorted/100000	–	–	–	–	–	109 ns	126 ns
order/get_miss/sorted/1000000	–	–	–	–	–	114 ns	117 ns
order/get_random/hashed/1000	–	–	–	–	–	10.9 ns	10.9 ns
order/get_random/hashed/100000	–	–	–	–	–	16.4 ns	16.6 ns
order/get_random/hashed/1000000	–	–	–	–	–	22.6 ns	23.1 ns
order/get_random/sorted/1000	–	–	–	–	–	99.3 ns	98.9 ns
order/get_random/sorted/100000	–	–	–	–	–	254 ns	247 ns
order/get_random/sorted/1000000	–	–	–	–	–	350 ns	336 ns
order/overwrite/hashed/1000	–	–	–	–	–	9.88 ns	10.1 ns
order/overwrite/hashed/100000	–	–	–	–	–	10 ns	10 ns
order/overwrite/hashed/1000000	–	–	–	–	–	10.2 ns	10.1 ns
order/overwrite/sorted/1000	–	–	–	–	–	61.8 ns	63.7 ns
order/overwrite/sorted/100000	–	–	–	–	–	101 ns	100 ns
order/overwrite/sorted/1000000	–	–	–	–	–	77.9 ns	77.4 ns
order/range_100/hashed/1000	–	–	–	–	–	8.72 µs	8.66 µs
order/range_100/hashed/100000	–	–	–	–	–	1.7 ms	1.75 ms
order/range_100/hashed/1000000	–	–	–	–	–	36.4 ms	32.8 ms
order/range_100/sorted/1000	–	–	–	–	–	458 ns	473 ns
order/range_100/sorted/100000	–	–	–	–	–	537 ns	549 ns
order/range_100/sorted/1000000	–	–	–	–	–	532 ns	538 ns